Prototyping – Übung 3

Themen: JavaScript Funktionen, SvelteKit-Projekt

Aufgabe – Taschenrechner (Basisanwendung)

In dieser Aufgabe erstellst du einen einfachen Taschenrechner mit JavaScript und SvelteKit.

Hinweise zur Funktionsweise eines Taschenrechners

Ein Taschenrechner (wie z.B. die «Rechner»-Anwendung von Windows) funktioniert wie folgt:

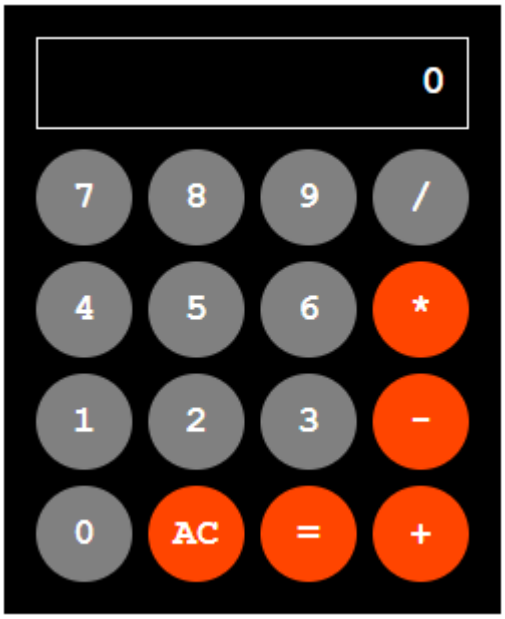


- Zahlen werden eingegeben, indem die Ziffern (0-9) einzeln getippt werden.
- Nachdem die erste Zahl eingegeben wurde, wird die Operation (plus, minus, mal, geteilt) gefolgt von der zweiten Zahl eingegeben.
- Danach kann entweder mit '=' das Endergebnis berechnet werden oder es kann eine weitere Operation gefolgt von einer weiteren Zahl eingeben werden.
  Beispiele von Eingabefolgen:
  - Um 56/2 zu berechnen:
    `5, 6, /, 2, =`
  - Um 56/2+87 zu berechnen:
    `5, 6, /, 2, +, 8, 7, =`
- In unserer Anwendung gehen wir davon aus, dass Zwischenresultate bei Rechnungen mit mehr als einem Operator sofort berechnet werden. Es gibt also kein «Punkt vor Strich».
  Beispiel: Bei der Rechnung 3-1*4 wird zuerst 3-1 berechnet und das Zwischenresultat (2) dann mit 4 multipliziert, was zum Endergebnis 8 führt.

a) Erstelle ein neues SvelteKit-Projekt mit dem Namen *calculator*. Öffne das Projekt in Visual Studio Code. (Eine Anleitung zum Erstellen von SvelteKit-Projekten findest du in den Vorlesungsunterlagen.)

Ersetze die Datei ***src/routes/+page.svelte*** durch die gleichnamige Datei ***+page.svelte***, die du von Moodle herunterladen kannst.

Starte die App mit `npm run dev`. Du solltest dann folgenden Taschenrechner im Browser sehen:



Der HTML- und `<style>`-Teil der Page sind bereits fertig implementiert.

HTML-Teil
Hier befinden sich die Elemente des Taschenrechners (Tasten, Anzeigefeld). Bei einem Klick auf einem Button wird jeweils eine JavaScript-Funktion aufgerufen, zum Beispiel:

Button	Funktionsaufruf
1	`setDigit(1)`
2	`setDigit(2)`
+	`setOperation("+")`
-	`setOperation("-")`
AC	`allClear()`
=	`calculate()`

`<style>`
Hier befindet sich das Stylesheet für das Layout und das Aussehen des Taschenrechners. Änderungen an dieser Datei sind in der Übung nicht gefordert. (Du darfst selbstverständlich beliebige Anpassungen vornehmen, wenn du das Aussehen des Taschenrechners ändern möchtest 😊.)

Hinweis: Du musst in dieser Übung nur den `<script>`-Teil anpassen.

Du benötigst für das Programm drei zusätzliche Variablen:

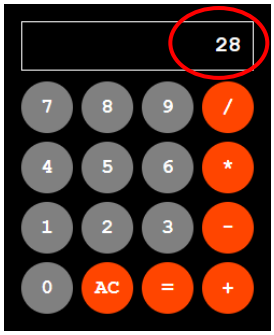
- **a**: Die erste Zahl, initialisiert mit dem Wert **0**
- **b**: Die zweite Zahl, initialisiert mit dem Wert **0**
- **`operation`**: Die gewählte Operation (`"+"`, `"-"`, `"*"`, `"/"`) als String. Diese Variable soll mit **`null`** initialisiert werden.

Ausserdem müssen wir uns die eingegebenen Zahlen und die Operation merken, damit wir am Ende den Ausdruck berechnen können. Implementiere dazu die folgenden Funktionen:

- **`setDigit(digit)`**
  In dieser Funktion passiert folgendes:
  i. Falls noch keine Operation eingegeben wurde (`operation` ist gleich `null`), wird das Argument `digit` in die Variable a gespeichert.
  ii. Falls bereits eine Operation eingegeben wurde (`operation` ist ungleich `null`), wird das Argument `digit` in die Variable b gespeichert.
- **`setOperation(op)`**
  In dieser Funktion wird die gewünschte Operation (z.B. «+») **als String** in der Variable `operation` gespeichert.
  Beispiel: `setOperation("+");`

Nun fehlt noch eine Funktion, um je nach gewählter Operation das Endergebnis zu berechnen:

- **`calculate()`**
  Diese Funktion nutzt je nach Wert der Variable `operation` eine der vier oben erstellten Funktionen, um das Endergebnis in die Variable `output` zu schreiben. Der Wert der Variable `output` wird dann via Databinding im Browser angezeigt:



Als letztes solltest du noch eine Funktion implementieren, mit der du den Rechner resetten kannst:

- **`allClear()`**
  Diese Funktion entspricht (je nach Taschenrechner) der Taste «AC» oder «C» und setzt die Werte von a, b, `operation` und `output` auf die Anfangswerte zurück, damit eine neue Rechnung begonnen werden kann.

Teste dein Programm: Starte die SvelteKit-App (`npm run dev`) und gib im Browser auf dem Taschenrechner eine Rechnung ein, z.B. 7 - 3. Die Rechnung selbst wird zwar noch nicht auf dem Display des Rechners angezeigt, dafür das Ergebnis, wenn du auf = klickst. Beachte, dass du jeweils auf AC klicken musst, wenn du eine neue Rechnung eingeben willst.

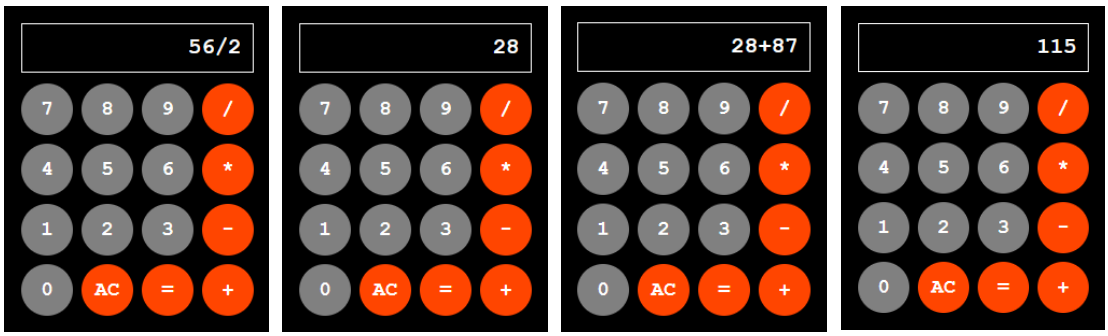
b) Nun soll die Rechnung im Display des Taschenrechners angezeigt werden. Nutze dazu die Variable `output`. Diese Variable muss bei jeder Eingabe von Ziffern und Operationen aktualisiert werden. Beispiel:

Eingabe	**Anzeige** (= Wert der Variable `output`)
	0 (Initialwert der Anzeige)
1	1
+	1+
2	1+2
=	3

Beachte den Spezialfall, dass ganz am Anfang der Rechnung eine 0 in der Anzeige steht. Wenn die erste Ziffer eingegeben wird, soll diese 0 zuerst gelöscht werden.

c) Der erste Schritt ist getan. Allerdings können wir bis jetzt nur einzelne Ziffern (0-9) eingeben und keine grösseren Zahlen. Erweitere also die Funktion **`setDigit(digit)`**, so dass der Wert von a bzw. b bei jeder Eingabe einer weiteren Ziffer aktualisiert wird.

d) Wie bei einem «richtigen» Taschenrechner soll nun mit dem Ergebnis weitergerechnet werden können. Beachte dazu folgendes Beispiel, wo zuerst 56/2 gerechnet und dann zum Ergebnis 87 addiert wird.



56/2 = +87 =

Damit dies funktioniert, musst du die Funktion **`calculate()`** auf geeignete Weise anpassen.

Teste dein Programm zum Beispiel mit folgenden Eingaben:

Eingabe	**Anzeige** (= Wert der Variable `output`)
	0 (Initialwert der Anzeige)
1	1
2	12
+	12+
3	12+3
4	12+34
=	46
*	46*
2	46*2
=	92

e) Nun ist es noch etwas unschön, dass wir jeweils auf = klicken müssen (damit die Funktion `calculate()` aufgerufen wird), um das Zwischenergebnis zu berechnen. Besser wäre es, wenn dies automatisch geschieht, wenn die zweite Operation eingegeben wird. Passe die Funktion `setOperation(op)` dementsprechend an.

Teste dein Programm zum Beispiel mit folgenden Eingaben. (Dies sind die gleichen Eingaben wie in der letzten Teilaufgabe ohne die erste Eingabe von = zur Berechnung des Zwischenergebnisses.)

Eingabe	**Anzeige** (= Wert der Variable `output`)
	0 (Initialwert der Anzeige)
1	1
2	12
+	12+
3	12+3
4	12+34
*	46*
2	46*2
=	92

Zusatzaufgabe (optional)

f) Der Rechner funktioniert nun, aber wenn mehrere Berechnungen im gleichen Programm ausgeführt werden, muss nach jeder Berechnung die Funktion **`allClear()`** (via Klick auf AC) aufgerufen werden. Dies kann aber automatisch gemacht werden, wenn auf einen Aufruf von `calculate()` direkt `setDigit(digit)` folgt.

Beachte dazu, wie z.B. der Windows-Rechner funktioniert.

Fall 1: Das Ergebnis einer Berechnung wird weiterverwendet, wenn anschliessend **eine Operation** eingegeben wird:

'56/2':	Klick auf '=': Ergebnis wird berechnet.	Eingabe von '+' gefolgt von '87': Ergebnis der vorherigen Schritt wird weiterverwendet.
		

Fall 2: Das Ergebnis einer Berechnung wird **nicht** weiterverwendet, wenn anschliessend **eine Zahl** eingegeben wird:

'56/2':	Klick auf '=': Ergebnis wird berechnet.	Eingabe von '87': Eine neue Rechnung wird begonnen.
		

Passe das Programm an, so dass der Aufruf von `allClear()` nach den einzelnen Tests nicht mehr nötig ist. Dazu sind Änderungen in mehreren Funktionen sowie eine zusätzliche Hilfsvariable nötig. Die Hilfsvariable wird benötigt, um sich den Zustand des Taschenrechners zu merken.

Teste dein Programm zum Beispiel mit folgenden Eingaben.

Eingabe	**Anzeige** (= Wert der Variable `output`)
	0 (Initialwert der Anzeige)
7	7
-	7-
3	7-3
=	4
1	1
3	13
*	13*
2	13*2
=	26
+	26+
5	26+5
-	31-
4	31-4
=	27
AC	0

Abgabe

Gib die Datei ***src/routes/+page.svelte*** auf Moodle ab.